

**Московский авиационный институт
Национальный исследовательский университет**

Лабораторная работа 5

Описание и моделирование последовательных устройств на языке Verilog HDL

Вариант 9

Выполнил:
студент группы М7О-404С-22
Смирнов А. П.

Проверил:
заведующий кафедрой 702,
профессор
Кривилёв А. В.

Дата: _____

Подпись: _____

Москва 2026

Содержание

1 Цель и задачи	4
2 Введение	4
3 Практическая часть	4
3.1 Синтез на основе словарного метода.....	4
4 Режим суммирования в коде Айкена.....	4
4.1 Карты Карно для кода Айкена	5
4.2 Минимизированные функции возбуждения	5
5 Режим вычитания в коде Грея.....	6
5.1 Карты Карно для кода Грея	6
5.2 Минимизированные функции возбуждения	7
6 Реверсивный режим	7
6.1 Функции возбуждения	8
7 Аналитические выражения в базисах Пирса Шеффера и Жегалкина	8
7.1 Базис Пирса	8
7.2 Базис Шеффера	8
7.3 Полиномы Жегалкина	9
8 Описание работы устройства на языке Verilog HDL	9
8.1 Структурное описание	9
8.1.1 Т-триггер	9
8.1.2 Суммирование в коде Айкена	9
8.1.3 Вычитание в коде Грея	10
8.1.4 Реверсивный счётчик Грея	10
8.2 Поведенческое описание	10
8.2.1 Тернарный оператор.....	10
8.2.2 Оператор case	11
8.2.3 Оператор if	12
8.2.4 Функция.....	13
8.3 Испытательный стенд и функциональное моделирование	15
9 Синтез устройства в Quartus Prime	18
9.1 MAX II EPM240T100C5	18
9.2 Cyclone IV EP4CE10E22C8.....	19
9.2.1 Верхний уровень платы	19
9.3 Экспериментальные исследования	22
10 Вывод.....	23

Приложение А Состав комплекта исходников	23
Приложение Б Соответствие листингов исходным файлам	24
Приложение В Порядок запуска и оформления результатов.....	24

1 Цель и задачи

Цель работы - изучить основные принципы построения последовательностных устройств, освоить их структурное и поведенческое описание на языке Verilog HDL и подготовить проекты для синтеза в Quartus Prime на ПЛИС семейств MAX II и Cyclone IV.

Требуется описать четырёхразрядный счётчик, выполнить синтез на основе словарного метода, получить аналитические выражения в базисах Пирса, Шеффера и Жегалкина, разработать параметризованное структурное описание и четыре поведенческих варианта, а также проверить их испытательным стендом.

№	Режим суммирования	Режим вычитания	Тип триггера	Студент
9	Айкена 2421	Грея	T	Смирнов А. П.

Таблица 1 - Индивидуальный вариант

2 Введение

Счётчик является конечным автоматом, состояние которого изменяется по активному фронту тактового сигнала. В данной работе в режиме суммирования используется самодополняющийся код Айкена 2421 для десятичных цифр от 0 до 9. В режиме вычитания применяется четырёхразрядный отражённый код Грея, в котором соседние кодовые слова отличаются ровно одним разрядом. Состояние хранится в четырёх T-триггерах.

Для T-триггера информационный вход определяется выражением $T_i = Q_i \oplus Q_i^+$. Поэтому после задания последовательности состояний сигналы возбуждения находятся непосредственно сравнением текущего и следующего кодовых слов.

3 Практическая часть

3.1 Синтез на основе словарного метода

Переход $Q_i \rightarrow Q_i^+$	T_i	Действие
0 $\rightarrow$ 0	0	Хранение нуля
0 $\rightarrow$ 1	1	Установка
1 $\rightarrow$ 0	1	Сброс
1 $\rightarrow$ 1	0	Хранение единицы

Таблица 2 - Словарь переходов T-триггера

$$T_i = Q_i \oplus Q_i^+$$

4 Режим суммирования в коде Айкена

i	Текущее состояние	Следующее состояние	$T_3T_2T_1T_0$	Код
0	0000	0001	0001	0
1	0001	0010	0011	1
2	0010	0011	0001	2
3	0011	0100	0111	3
4	0100	1011	1111	4

i	Текущее состояние	Следующее состояние	$T_3T_2T_1T_0$	Код
5	1011	1100	0111	11
6	1100	1101	0001	12
7	1101	1110	0011	13
8	1110	1111	0001	14
9	1111	0000	1111	15

Таблица 3 - Таблица состояний: режим суммирования в коде айкена

Код Айкена 2421 использует десять кодовых слов: 0000, 0001, 0010, 0011, 0100, 1011, 1100, 1101, 1110 и 1111. Остальные шесть комбинаций не соответствуют десятичным цифрам и при минимизации отмечаются как безразличные условия.

4.1 Карты Карно для кода Айкена

$B_3B_2 \setminus B_1B_0$	00	01	11	10
00	1	1	1	1
01	1	-	-	-
11	1	1	1	1
10	-	-	1	-

Таблица 4 - Карта Карно для T_0

$B_3B_2 \setminus B_1B_0$	00	01	11	10
00	0	1	1	0
01	1	-	-	-
11	0	1	1	0
10	-	-	1	-

Таблица 5 - Карта Карно для T_1

$B_3B_2 \setminus B_1B_0$	00	01	11	10
00	0	0	1	0
01	1	-	-	-
11	0	0	1	0
10	-	-	1	-

Таблица 6 - Карта Карно для T_2

$B_3B_2 \setminus B_1B_0$	00	01	11	10
00	0	0	0	0
01	1	-	-	-
11	0	0	1	0
10	-	-	0	-

Таблица 7 - Карта Карно для T_3

4.2 Минимизированные функции возбуждения

$$T_0 = 1$$

$$T_1 = B_0 \vee (\neg B_3 \wedge B_2)$$

$$T_2 = (B_1 \wedge B_0) \vee (\neg B_3 \wedge B_2)$$

$$T_3 = (B_2 \wedge B_1 \wedge B_0) \vee (\neg B_3 \wedge B_2)$$

Минимизированные функции обеспечивают требуемый десятичный цикл. Из любого неиспользуемого состояния структурная схема возвращается в допустимый цикл не более чем за четыре такта.

5 Режим вычитания в коде Грея

i	Текущее состояние	Следующее состояние	$T_3T_2T_1T_0$	Код
0	0000	1000	1000	0
1	0001	0000	0001	1
2	0011	0001	0010	3
3	0010	0011	0001	2
4	0110	0010	0100	6
5	0111	0110	0001	7
6	0101	0111	0010	5
7	0100	0101	0001	4
8	1100	0100	1000	12
9	1101	1100	0001	13
10	1111	1101	0010	15
11	1110	1111	0001	14
12	1010	1110	0100	10
13	1011	1010	0001	11
14	1001	1011	0010	9
15	1000	1001	0001	8

Таблица 8 - Таблица состояний: режим вычитания в коде Грея

5.1 Карты Карно для кода Грея

$V_3V_2 \setminus V_1V_0$	00	01	11	10
00	0	1	0	1
01	1	0	1	0
11	0	1	0	1
10	1	0	1	0

Таблица 9 - Карта Карно для T_0

$V_3V_2 \setminus V_1V_0$	00	01	11	10
00	0	0	1	0
01	0	1	0	0
11	0	0	1	0
10	0	1	0	0

Таблица 10 - Карта Карно для T_1

$B_3B_2 \setminus B_1B_0$	00	01	11	10
00	0	0	0	0
01	0	0	0	1
11	0	0	0	0
10	0	0	0	1

Таблица 11 - Карта Карно для T2

$B_3B_2 \setminus B_1B_0$	00	01	11	10
00	1	0	0	0
01	0	0	0	0
11	1	0	0	0
10	0	0	0	0

Таблица 12 - Карта Карно для T3

5.2 Минимизированные функции возбуждения

$$P = B_3 \oplus B_2 \oplus B_1 \oplus B_0$$

$$T_0 \downarrow = P$$

$$T_1 \downarrow = B_0 \wedge (B_3 \oplus B_2 \oplus B_1)$$

$$T_2 \downarrow = \neg B_0 \wedge B_1 \wedge (B_3 \oplus B_2)$$

$$T_3 \downarrow = \neg B_1 \wedge \neg B_0 \wedge \neg(B_3 \oplus B_2)$$

6 Реверсивный режим

i	dir	Текущее состояние	Следующее состояние	$T_3T_2T_1T_0$
0	0	0000	0001	0001
1	0	0001	0011	0010
2	0	0011	0010	0001
3	0	0010	0110	0100
4	0	0110	0111	0001
5	0	0111	0101	0010
6	0	0101	0100	0001
7	0	0100	1100	1000
8	0	1100	1101	0001
9	0	1101	1111	0010
10	0	1111	1110	0001
11	0	1110	1010	0100
12	0	1010	1011	0001
13	0	1011	1001	0010
14	0	1001	1000	0001
15	0	1000	0000	1000
16	1	0000	1000	1000
17	1	0001	0000	0001
18	1	0011	0001	0010

i	dir	Текущее состояние	Следующее состояние	T ₃ T ₂ T ₁ T ₀
19	1	0010	0011	0001
20	1	0110	0010	0100
21	1	0111	0110	0001
22	1	0101	0111	0010
23	1	0100	0101	0001
24	1	1100	0100	1000
25	1	1101	1100	0001
26	1	1111	1101	0010
27	1	1110	1111	0001
28	1	1010	1110	0100
29	1	1011	1010	0001
30	1	1001	1011	0010
31	1	1000	1001	0001

Таблица 13 - Таблица состояний реверсивного счётчика Грея

6.1 Функции возбуждения

$$\begin{aligned}
 T_0 \uparrow &= \neg P \\
 T_1 \uparrow &= B_0 \wedge \neg(B_3 \oplus B_2 \oplus B_1) \\
 T_2 \uparrow &= \neg B_0 \wedge B_1 \wedge \neg(B_3 \oplus B_2) \\
 T_3 \uparrow &= \neg B_1 \wedge \neg B_0 \wedge (B_3 \oplus B_2) \\
 T_i &= (\neg \text{dir} \wedge T_i \uparrow) \vee (\text{dir} \wedge T_i \downarrow)
 \end{aligned}$$

Сигнал dir является внешним входом выбора направления и не хранится в отдельном триггере. При dir = 0 выбираются функции счёта вверх, при dir = 1 - функции счёта вниз.

7 Аналитические выражения в базисах Пирса Шеффера и Жегалкина

Преобразование выполняется для минимизированных функций режима суммирования в коде Айкена. Для компактной записи вводятся вспомогательные операции, каждая из которых содержит только один разрешённый двухместный оператор.

7.1 Базис Пирса

$$\begin{aligned}
 N \downarrow (x) &= x \downarrow x \\
 A \downarrow (x, y) &= (x \downarrow x) \downarrow (y \downarrow y) \\
 O \downarrow (x, y) &= (x \downarrow y) \downarrow (x \downarrow y) \\
 T_0 &= 1 \\
 T_1 &= O \downarrow (B_0, A \downarrow (N \downarrow (B_3), B_2)) \\
 T_2 &= O \downarrow (A \downarrow (B_1, B_0), A \downarrow (N \downarrow (B_3), B_2)) \\
 T_3 &= O \downarrow (A \downarrow (A \downarrow (B_2, B_1), B_0), A \downarrow (N \downarrow (B_3), B_2))
 \end{aligned}$$

7.2 Базис Шеффера

$$\begin{aligned}
 N \uparrow (x) &= x \uparrow x \\
 A \uparrow (x, y) &= (x \uparrow y) \uparrow (x \uparrow y)
 \end{aligned}$$

$$\begin{aligned}
O \uparrow (x, y) &= (x \uparrow x) \uparrow (y \uparrow y) \\
T_0 &= 1 \\
T_1 &= O \uparrow (B_0, A \uparrow (N \uparrow (B_3), B_2)) \\
T_2 &= O \uparrow (A \uparrow (B_1, B_0), A \uparrow (N \uparrow (B_3), B_2)) \\
T_3 &= O \uparrow (A \uparrow (A \uparrow (B_2, B_1), B_0), A \uparrow (N \uparrow (B_3), B_2))
\end{aligned}$$

7.3 Полиномы Жегалкина

$$\begin{aligned}
T_0 &= 1 \\
T_1 &= B_0 \oplus B_2 \oplus B_2B_0 \oplus B_3B_2 \oplus B_3B_2B_0 \\
T_2 &= B_1B_0 \oplus B_2 \oplus B_2B_1B_0 \oplus B_3B_2 \oplus B_3B_2B_1B_0 \\
T_3 &= B_2 \oplus B_3B_2 \oplus B_3B_2B_1B_0
\end{aligned}$$

8 Описание работы устройства на языке Verilog HDL

8.1 Структурное описание

Структурный модуль содержит четыре экземпляра Т-триггера. Параметр MODE выбирает режим: 0 - Айкен вверх, 1 - Грей вниз, 2 - реверсивный Грей. Непрерывные присваивания реализуют минимизированные функции возбуждения.

8.1.1 Т-триггер

Листинг 1 - Полный текст файла src/t_ff.v

```

`timescale 1ns/1ps

// Отчет: раздел 8.1.1, листинг 1.
// Модуль t_ff - один Т-триггер с синхронным разрешением enable.

module t_ff (
    input wire clock,
    input wire enable,
    input wire t,
    output reg q = 1'b0
);
    always @(posedge clock) begin
        if (enable && t)
            q <= ~q;
    end
endmodule

```

8.1.2 Суммирование в коде Айкена

При MODE = 0 выбирается вектор t_aiken_up, полученный из минимизированных функций раздела 4. Четыре экземпляра t_ff образуют регистр состояния.

Листинг 2 - Полный текст файла src/counter_structural.v

```

`timescale 1ns/1ps

// Отчет: разделы 8.1.2-8.1.4, листинг 2.
// Модуль counter_structural - структурный счетчик на четырех t_ff.
// MODE = 0: суммирование, код Айкена 2421.
// MODE = 1: вычитание, отраженный код Грея.
// MODE = 2: реверсивный счет в коде Грея; dir=0 вверх, dir=1 вниз.
module counter_structural #(
    parameter integer MODE = 0
) (
    input wire clock,
    input wire enable,
    input wire dir,
    output wire [3:0] q
);

```

```

wire parity;
wire [3:0] t_aiken_up;
wire [3:0] t_gray_down;
wire [3:0] t_gray_up;
wire [3:0] t_selected;

assign parity = q[3] ^ q[2] ^ q[1] ^ q[0];

// Минимизированные функции возбуждения Т-триггеров для кода Айкена.
assign t_aiken_up[0] = 1'b1;
assign t_aiken_up[1] = q[0] | (~q[3] & q[2]);
assign t_aiken_up[2] = (q[1] & q[0]) | (~q[3] & q[2]);
assign t_aiken_up[3] = (q[2] & q[1] & q[0]) | (~q[3] & q[2]);

// Счет вниз в отраженном коде Грея.
assign t_gray_down[0] = parity;
assign t_gray_down[1] = q[0] & (q[3] ^ q[2] ^ q[1]);
assign t_gray_down[2] = ~q[0] & q[1] & (q[3] ^ q[2]);
assign t_gray_down[3] = ~q[1] & ~q[0] & ~(q[3] ^ q[2]);

// Счет вверх в отраженном коде Грея.
assign t_gray_up[0] = ~parity;
assign t_gray_up[1] = q[0] & ~(q[3] ^ q[2] ^ q[1]);
assign t_gray_up[2] = ~q[0] & q[1] & ~(q[3] ^ q[2]);
assign t_gray_up[3] = ~q[1] & ~q[0] & (q[3] ^ q[2]);

generate
  if (MODE == 0) begin : select_aiken_up
    assign t_selected = t_aiken_up;
  end else if (MODE == 1) begin : select_gray_down
    assign t_selected = t_gray_down;
  end else begin : select_gray_reversible
    assign t_selected = dir ? t_gray_down : t_gray_up;
  end
endgenerate

genvar i;
generate
  for (i = 0; i < 4; i = i + 1) begin : make_t_ff
    t_ff bit_ff (
      .clock (clock),
      .enable (enable),
      .t      (t_selected[i]),
      .q      (q[i])
    );
  end
endgenerate
endmodule

```

8.1.3 Вычитание в коде Грея

Тот же листинг 2 реализует вычитание при $MODE = 1$. В этом режиме выбирается вектор `t_gray_down`, соответствующий таблице состояний раздела 5.

8.1.4 Реверсивный счётчик Грея

При $MODE = 2$ направление задаёт вход `dir`: ноль выбирает `t_gray_up`, единица - `t_gray_down`. Дополнительный триггер направления не используется.

8.2 Поведенческое описание

Подготовлены четыре эквивалентные реализации. Во всех вариантах сначала формируется следующее состояние, затем вычисляется вектор $T = Q \oplus Q^+$ и используются те же Т-триггеры. Ниже полностью приведены все четыре модуля файла `counter_behavioral.v`.

8.2.1 Тернарный оператор

Листинг 3 - Полный модуль `counter_beh_ternary` из `src/counter_behavioral.v`

```

module counter_beh_ternary #(
  parameter integer MODE = 0
) (

```

```

input wire clock,
input wire enable,
input wire dir,
output wire [3:0] q
);
wire [3:0] next_aiken;
wire [3:0] next_gray_down;
wire [3:0] next_gray_up;
wire [3:0] next_state;
wire [3:0] t;

assign next_aiken =
  (q == 4'b0000) ? 4'b0001 :
  (q == 4'b0001) ? 4'b0010 :
  (q == 4'b0010) ? 4'b0011 :
  (q == 4'b0011) ? 4'b0100 :
  (q == 4'b0100) ? 4'b1011 :
  (q == 4'b1011) ? 4'b1100 :
  (q == 4'b1100) ? 4'b1101 :
  (q == 4'b1101) ? 4'b1110 :
  (q == 4'b1110) ? 4'b1111 :
  (q == 4'b1111) ? 4'b0000;

assign next_gray_down =
  (q == 4'b0000) ? 4'b1000 :
  (q == 4'b0001) ? 4'b0000 :
  (q == 4'b0011) ? 4'b0001 :
  (q == 4'b0010) ? 4'b0011 :
  (q == 4'b0110) ? 4'b0010 :
  (q == 4'b0111) ? 4'b0110 :
  (q == 4'b0101) ? 4'b0111 :
  (q == 4'b0100) ? 4'b0101 :
  (q == 4'b1100) ? 4'b0100 :
  (q == 4'b1101) ? 4'b1100 :
  (q == 4'b1111) ? 4'b1101 :
  (q == 4'b1110) ? 4'b1111 :
  (q == 4'b1010) ? 4'b1110 :
  (q == 4'b1011) ? 4'b1010 :
  (q == 4'b1001) ? 4'b1011 : 4'b1001;

assign next_gray_up =
  (q == 4'b0000) ? 4'b0001 :
  (q == 4'b0001) ? 4'b0011 :
  (q == 4'b0011) ? 4'b0010 :
  (q == 4'b0010) ? 4'b0110 :
  (q == 4'b0110) ? 4'b0111 :
  (q == 4'b0111) ? 4'b0101 :
  (q == 4'b0101) ? 4'b0100 :
  (q == 4'b0100) ? 4'b1100 :
  (q == 4'b1100) ? 4'b1101 :
  (q == 4'b1101) ? 4'b1111 :
  (q == 4'b1111) ? 4'b1110 :
  (q == 4'b1110) ? 4'b1010 :
  (q == 4'b1010) ? 4'b1011 :
  (q == 4'b1011) ? 4'b1001 :
  (q == 4'b1001) ? 4'b1000 : 4'b0000;

assign next_state = (MODE == 0) ? next_aiken :
  (MODE == 1) ? next_gray_down :
  (dir ? next_gray_down : next_gray_up);
assign t = q ^ next_state;

genvar i;
generate
  for (i = 0; i < 4; i = i + 1) begin : make_t_ff
    t_ff bit_ff(clock, enable, t[i], q[i]);
  end
endgenerate
endmodule

```

8.2.2 Оператор case

Листинг 4 - Полный модуль counter_beh_case из src/counter_behavioral.v

```

module counter_beh_case #(
  parameter integer MODE = 0
) (
  input wire clock,

```

```

input wire enable,
input wire dir,
output wire [3:0] q
);
reg [3:0] next_state;
wire [3:0] t;

always @(*) begin
    next_state = 4'b0000;
    if (MODE == 0) begin
        case (q)
            4'b0000: next_state = 4'b0001;
            4'b0001: next_state = 4'b0010;
            4'b0010: next_state = 4'b0011;
            4'b0011: next_state = 4'b0100;
            4'b0100: next_state = 4'b0101;
            4'b0101: next_state = 4'b0110;
            4'b0110: next_state = 4'b0111;
            4'b0111: next_state = 4'b1000;
            default: next_state = 4'b0000;
        endcase
    end else if ((MODE == 1) || dir) begin
        case (q)
            4'b0000: next_state = 4'b1000;
            4'b0001: next_state = 4'b0000;
            4'b0011: next_state = 4'b0001;
            4'b0010: next_state = 4'b0011;
            4'b0110: next_state = 4'b0010;
            4'b0111: next_state = 4'b0110;
            4'b0101: next_state = 4'b0111;
            4'b0100: next_state = 4'b0101;
            4'b1100: next_state = 4'b0100;
            4'b1101: next_state = 4'b1100;
            4'b1111: next_state = 4'b1101;
            4'b1110: next_state = 4'b1111;
            4'b1010: next_state = 4'b1110;
            4'b1011: next_state = 4'b1010;
            4'b1001: next_state = 4'b1011;
            4'b1000: next_state = 4'b1001;
            default: next_state = 4'b0000;
        endcase
    end else begin
        case (q)
            4'b0000: next_state = 4'b0001;
            4'b0001: next_state = 4'b0011;
            4'b0011: next_state = 4'b0010;
            4'b0010: next_state = 4'b0110;
            4'b0110: next_state = 4'b0111;
            4'b0111: next_state = 4'b0101;
            4'b0101: next_state = 4'b0100;
            4'b0100: next_state = 4'b1100;
            4'b1100: next_state = 4'b1101;
            4'b1101: next_state = 4'b1111;
            4'b1111: next_state = 4'b1110;
            4'b1110: next_state = 4'b1010;
            4'b1010: next_state = 4'b1011;
            4'b1011: next_state = 4'b1001;
            4'b1001: next_state = 4'b1000;
            default: next_state = 4'b0000;
        endcase
    end
end

assign t = q ^ next_state;
genvar i;
generate
    for (i = 0; i < 4; i = i + 1) begin : make_t_ff
        t_ff bit_ff(clock, enable, t[i], q[i]);
    end
endgenerate
endmodule

```

8.2.3 Оператор if

Листинг 5 - Полный модуль counter_beh_if из src/counter_behavioral.v

```

module counter_beh_if #(
    parameter integer MODE = 0
) (
    input wire clock,
    input wire enable,
    input wire dir,
    output wire [3:0] q
);
    reg [3:0] next_state;
    wire [3:0] t;

    always @(*) begin
        next_state = 4'b0000;
        if (MODE == 0) begin
            if (q == 4'b0000) next_state = 4'b0001;
            else if (q == 4'b0001) next_state = 4'b0010;
            else if (q == 4'b0010) next_state = 4'b0011;
            else if (q == 4'b0011) next_state = 4'b0100;
            else if (q == 4'b0100) next_state = 4'b0101;
            else if (q == 4'b0101) next_state = 4'b0110;
            else if (q == 4'b0110) next_state = 4'b0111;
            else if (q == 4'b0111) next_state = 4'b1000;
            else if (q == 4'b1000) next_state = 4'b1001;
            else if (q == 4'b1001) next_state = 4'b1010;
            else if (q == 4'b1010) next_state = 4'b1011;
            else if (q == 4'b1011) next_state = 4'b1100;
            else if (q == 4'b1100) next_state = 4'b1101;
            else if (q == 4'b1101) next_state = 4'b1110;
            else if (q == 4'b1110) next_state = 4'b1111;
            else if (q == 4'b1111) next_state = 4'b0000;
        end else if ((MODE == 1) || dir) begin
            if (q == 4'b0000) next_state = 4'b1000;
            else if (q == 4'b1000) next_state = 4'b1001;
            else if (q == 4'b1001) next_state = 4'b1011;
            else if (q == 4'b1011) next_state = 4'b1010;
            else if (q == 4'b1010) next_state = 4'b1110;
            else if (q == 4'b1110) next_state = 4'b1111;
            else if (q == 4'b1111) next_state = 4'b1101;
            else if (q == 4'b1101) next_state = 4'b1100;
            else if (q == 4'b1100) next_state = 4'b0100;
            else if (q == 4'b0100) next_state = 4'b0101;
            else if (q == 4'b0101) next_state = 4'b0111;
            else if (q == 4'b0111) next_state = 4'b0110;
            else if (q == 4'b0110) next_state = 4'b0010;
            else if (q == 4'b0010) next_state = 4'b0011;
            else if (q == 4'b0011) next_state = 4'b0001;
            else if (q == 4'b0001) next_state = 4'b0000;
        end else begin
            if (q == 4'b0000) next_state = 4'b0001;
            else if (q == 4'b0001) next_state = 4'b0011;
            else if (q == 4'b0011) next_state = 4'b0010;
            else if (q == 4'b0010) next_state = 4'b0110;
            else if (q == 4'b0110) next_state = 4'b0111;
            else if (q == 4'b0111) next_state = 4'b0101;
            else if (q == 4'b0101) next_state = 4'b0100;
            else if (q == 4'b0100) next_state = 4'b1100;
            else if (q == 4'b1100) next_state = 4'b1101;
            else if (q == 4'b1101) next_state = 4'b1111;
            else if (q == 4'b1111) next_state = 4'b1110;
            else if (q == 4'b1110) next_state = 4'b1010;
            else if (q == 4'b1010) next_state = 4'b1011;
            else if (q == 4'b1011) next_state = 4'b1001;
            else if (q == 4'b1001) next_state = 4'b1000;
            else if (q == 4'b1000) next_state = 4'b0000;
        end
    end

    assign t = q ^ next_state;
    genvar i;
    generate
        for (i = 0; i < 4; i = i + 1) begin : make_t_ff
            t_ff bit_ff(clock, enable, t[i], q[i]);
        end
    endgenerate
endmodule

```

8.2.4 Функция

Листинг 6 - Полный модуль counter_beh_function из src/counter_behavioral.v

```

module counter_beh_function #(
    parameter integer MODE = 0
) (
    input wire clock,
    input wire enable,

```

```

input wire dir,
output wire [3:0] q
);

wire [3:0] next_state;
wire [3:0] t;

function [3:0] next_aiken;
input [3:0] state;
begin
case (state)
4'h0: next_aiken = 4'h1;
4'h1: next_aiken = 4'h2;
4'h2: next_aiken = 4'h3;
4'h3: next_aiken = 4'h4;
4'h4: next_aiken = 4'h8;
4'h8: next_aiken = 4'hC;
4'hC: next_aiken = 4'hD;
4'hD: next_aiken = 4'hE;
4'hE: next_aiken = 4'hF;
4'hF: next_aiken = 4'h0;
default: next_aiken = 4'h0;
endcase
end
endfunction

function [3:0] next_gray_down;
input [3:0] state;
begin
case (state)
4'h0: next_gray_down = 4'h8;
4'h8: next_gray_down = 4'h9;
4'h9: next_gray_down = 4'hB;
4'hB: next_gray_down = 4'hA;
4'hA: next_gray_down = 4'hE;
4'hE: next_gray_down = 4'hF;
4'hF: next_gray_down = 4'hD;
4'hD: next_gray_down = 4'hC;
4'hC: next_gray_down = 4'h4;
4'h4: next_gray_down = 4'h5;
4'h5: next_gray_down = 4'h7;
4'h7: next_gray_down = 4'h6;
4'h6: next_gray_down = 4'h2;
4'h2: next_gray_down = 4'h3;
4'h3: next_gray_down = 4'h1;
4'h1: next_gray_down = 4'h0;
default: next_gray_down = 4'h0;
endcase
end
endfunction

function [3:0] next_gray_up;
input [3:0] state;
begin
case (state)
4'h0: next_gray_up = 4'h1;
4'h1: next_gray_up = 4'h3;
4'h3: next_gray_up = 4'h2;
4'h2: next_gray_up = 4'h6;
4'h6: next_gray_up = 4'h7;
4'h7: next_gray_up = 4'h5;
4'h5: next_gray_up = 4'h4;
4'h4: next_gray_up = 4'hC;
4'hC: next_gray_up = 4'hD;
4'hD: next_gray_up = 4'hF;
4'hF: next_gray_up = 4'hE;
4'hE: next_gray_up = 4'hA;
4'hA: next_gray_up = 4'hB;
4'hB: next_gray_up = 4'h9;
4'h9: next_gray_up = 4'h8;
4'h8: next_gray_up = 4'h0;
default: next_gray_up = 4'h0;
endcase
end
endfunction

assign next_state = (MODE == 0) ? next_aiken(q) :
(MODE == 1) ? next_gray_down(q) :
(dir ? next_gray_down(q) : next_gray_up(q));
assign t = q ^ next_state;

```

```

genvar i;
generate
  for (i = 0; i < 4; i = i + 1) begin : make_t_ff
    t_ff bit_ff(clock, enable, t[i], q[i]);
  end
endgenerate
endmodule

```

8.3 Испытательный стенд и функциональное моделирование

Самопроверяющийся стенд `tb_all.v` одновременно запускает структурный модуль и все четыре поведенческих варианта. После каждого положительного фронта результаты сравниваются с эталонными функциями переходов. При отсутствии ошибок в окне Transcript выводится сообщение PASS.

Листинг 7 - Полный текст файла `tb/tb_all.v`

```

`timescale 1ns/1ps

// Отчет: раздел 8.3, листинг 7.
// Самопроверяющийся стенд для структурного и четырех поведенческих описаний.

module tb_all;
  reg clock = 1'b0;
  reg enable = 1'b1;
  reg dir = 1'b0;
  integer errors = 0;
  integer cycle;
  reg [3:0] expected_aiken = 4'h0;
  reg [3:0] expected_gray_down = 4'h0;
  reg [3:0] expected_reverse = 4'h0;

  wire [3:0] s_aiken, s_gray_down, s_reverse;
  wire [3:0] t_aiken, c_aiken, i_aiken, f_aiken;
  wire [3:0] t_gray_down, c_gray_down, i_gray_down, f_gray_down;
  wire [3:0] t_reverse, c_reverse, i_reverse, f_reverse;

  counter_structural #(.MODE(0)) u_s_aiken(clock, enable, dir, s_aiken);
  counter_structural #(.MODE(1)) u_s_gray_down(clock, enable, dir, s_gray_down);
  counter_structural #(.MODE(2)) u_s_reverse(clock, enable, dir, s_reverse);

  counter_beh_ternary #(.MODE(0)) u_t_aiken(clock, enable, dir, t_aiken);
  counter_beh_case #(.MODE(0)) u_c_aiken(clock, enable, dir, c_aiken);
  counter_beh_if #(.MODE(0)) u_i_aiken(clock, enable, dir, i_aiken);
  counter_beh_function #(.MODE(0)) u_f_aiken(clock, enable, dir, f_aiken);

  counter_beh_ternary #(.MODE(1)) u_t_gray_down(clock, enable, dir, t_gray_down);
  counter_beh_case #(.MODE(1)) u_c_gray_down(clock, enable, dir, c_gray_down);
  counter_beh_if #(.MODE(1)) u_i_gray_down(clock, enable, dir, i_gray_down);
  counter_beh_function #(.MODE(1)) u_f_gray_down(clock, enable, dir, f_gray_down);

  counter_beh_ternary #(.MODE(2)) u_t_reverse(clock, enable, dir, t_reverse);
  counter_beh_case #(.MODE(2)) u_c_reverse(clock, enable, dir, c_reverse);
  counter_beh_if #(.MODE(2)) u_i_reverse(clock, enable, dir, i_reverse);
  counter_beh_function #(.MODE(2)) u_f_reverse(clock, enable, dir, f_reverse);

  always #5 clock = ~clock;

  function [3:0] next_aiken;
    input [3:0] state;
    begin
      case (state)
        4'h0: next_aiken = 4'h1;
        4'h1: next_aiken = 4'h2;
        4'h2: next_aiken = 4'h3;
        4'h3: next_aiken = 4'h4;
        4'h4: next_aiken = 4'hB;
        4'hB: next_aiken = 4'hC;
        4'hC: next_aiken = 4'hD;
        4'hD: next_aiken = 4'hE;
        4'hE: next_aiken = 4'hF;
        4'hF: next_aiken = 4'h0;
        default: next_aiken = 4'h0;
      endcase
    end
  endfunction

```

```

        endcase
    end
endfunction

function [3:0] next_gray_down;
    input [3:0] state;
    begin
        case (state)
            4'h0: next_gray_down = 4'h8;
            4'h8: next_gray_down = 4'h9;
            4'h9: next_gray_down = 4'hB;
            4'hB: next_gray_down = 4'hA;
            4'hA: next_gray_down = 4'hE;
            4'hE: next_gray_down = 4'hF;
            4'hF: next_gray_down = 4'hD;
            4'hD: next_gray_down = 4'hC;
            4'hC: next_gray_down = 4'h4;
            4'h4: next_gray_down = 4'h5;
            4'h5: next_gray_down = 4'h7;
            4'h7: next_gray_down = 4'h6;
            4'h6: next_gray_down = 4'h2;
            4'h2: next_gray_down = 4'h3;
            4'h3: next_gray_down = 4'h1;
            default: next_gray_down = 4'h0;
        endcase
    end
endfunction

function [3:0] next_gray_up;
    input [3:0] state;
    begin
        case (state)
            4'h0: next_gray_up = 4'h1;
            4'h1: next_gray_up = 4'h3;
            4'h3: next_gray_up = 4'h2;
            4'h2: next_gray_up = 4'h6;
            4'h6: next_gray_up = 4'h7;
            4'h7: next_gray_up = 4'h5;
            4'h5: next_gray_up = 4'h4;
            4'h4: next_gray_up = 4'hC;
            4'hC: next_gray_up = 4'hD;
            4'hD: next_gray_up = 4'hF;
            4'hF: next_gray_up = 4'hE;
            4'hE: next_gray_up = 4'hA;
            4'hA: next_gray_up = 4'hB;
            4'hB: next_gray_up = 4'h9;
            4'h9: next_gray_up = 4'h8;
            default: next_gray_up = 4'h0;
        endcase
    end
endfunction

task check_all;
    begin
        if ((s_aiken != expected_aiken) ||
            (t_aiken != expected_aiken) ||
            (c_aiken != expected_aiken) ||
            (i_aiken != expected_aiken) ||
            (f_aiken != expected_aiken)) begin
            $display("ERROR Aiken cycle=%0d expected=%h", cycle, expected_aiken);
            errors = errors + 1;
        end
        if ((s_gray_down != expected_gray_down) ||
            (t_gray_down != expected_gray_down) ||
            (c_gray_down != expected_gray_down) ||
            (i_gray_down != expected_gray_down) ||
            (f_gray_down != expected_gray_down)) begin
            $display("ERROR Gray down cycle=%0d expected=%h", cycle, expected_gray_down);
            errors = errors + 1;
        end
        if ((s_reverse != expected_reverse) ||
            (t_reverse != expected_reverse) ||
            (c_reverse != expected_reverse) ||
            (i_reverse != expected_reverse) ||
            (f_reverse != expected_reverse)) begin
            $display("ERROR reverse cycle=%0d dir=%b expected=%h", cycle, dir, expected_reverse);
            errors = errors + 1;
        end
    end
endtask

```

```

initial begin
    $dumpfile("lab5.vcd");
    $dumpvars(0, tb_all);

    for (cycle = 1; cycle <= 20; cycle = cycle + 1) begin
        @(posedge clock);
        expected_aiken = next_aiken(expected_aiken);
        expected_gray_down = next_gray_down(expected_gray_down);
        expected_reverse = next_gray_up(expected_reverse);
        #1 check_all;
    end

    dir = 1'b1;
    for (cycle = 21; cycle <= 40; cycle = cycle + 1) begin
        @(posedge clock);
        expected_aiken = next_aiken(expected_aiken);
        expected_gray_down = next_gray_down(expected_gray_down);
        expected_reverse = next_gray_down(expected_reverse);
        #1 check_all;
    end

    enable = 1'b0;
    @(posedge clock);
    #1 check_all;

    if (errors == 0)
        $display("PASS: all structural and behavioral variants are correct");
    else
        $display("FAIL: %0d mismatches", errors);
    $finish;
end
endmodule

```

Листинг 8 - Полный сценарий sim/run_sim.do

```

transcript on
# Отчет: раздел 8.3, листинг 8.
# Запускать из каталога sim командой: do run_sim.do
if {[file exists work]} {vdel -lib work -all}
vlib work
vmap work work
vlog -work work ../src/t_ff.v
vlog -work work ../src/counter_structural.v
vlog -work work ../src/counter_behavioral.v
vlog -work work ../tb/tb_all.v
vsim -voptargs=+acc work.tb_all
add wave -divider {Inputs}
add wave sim:/tb_all/clock
add wave sim:/tb_all/enable
add wave sim:/tb_all/dir
add wave -divider {Structural}
add wave -radix binary sim:/tb_all/s_aiken
add wave -radix binary sim:/tb_all/s_gray_down
add wave -radix binary sim:/tb_all/s_reverse
add wave -divider {Behavioral function}
add wave -radix binary sim:/tb_all/f_aiken
add wave -radix binary sim:/tb_all/f_gray_down
add wave -radix binary sim:/tb_all/f_reverse
run -all
wave zoom full

```

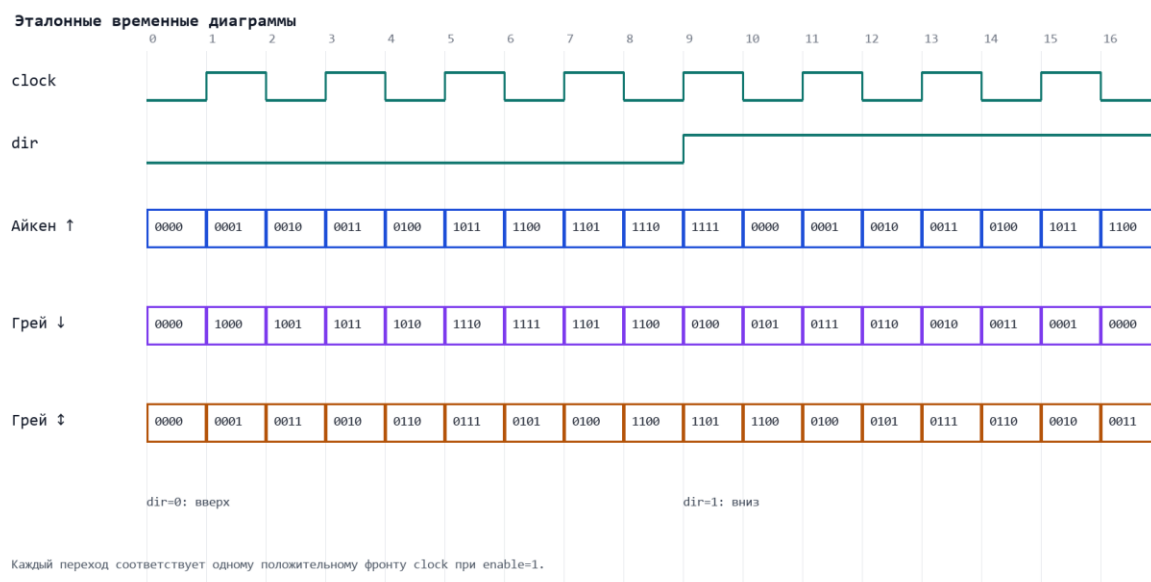


Рисунок 1 - Эталонные временные диаграммы проверяемых режимов

На диаграмме показаны ожидаемые последовательности. Два следующих поля предназначены для фактических результатов ModelSim или Questa.

ВСТАВИТЬ СЮДА РИСУНОК 2

Скриншот после do run_sim.do. Должна быть видна строка PASS: all structural and behavioral variants are correct.

Рисунок 2 - Результат самопроверки в окне Transcript

ВСТАВИТЬ СЮДА РИСУНОК 3

Окно Wave после wave zoom full; показать clock, enable, dir и шины s_* и f_* в двоичном виде.

Рисунок 3 - Временные диаграммы структурного и поведенческих описаний

9 Синтез устройства в Quartus Prime

9.1 MAX II EPM240T100C5

Проект quartus/maxii/lab5_maxii.qpf настроен на устройство EPM240T100C5. Верхний уровень synth_top использует структурное описание счётчика Айкена. Тактовое ограничение 20 нс задано в lab5_maxii.sdc. Полная компиляция в Quartus Prime 15.1 завершена успешно.

Параметр	Результат
Статус	Успешно, 0 ошибок, 7 предупреждений
Логические элементы	4 из 240 (2 %)
Выводы	6 из 80 (8 %)
Setup slack	17,138 нс
Hold slack	1,978 нс

Параметр	Результат
Файл программирования	output_files/lab5_maxii.pof

Таблица 14 - Результаты синтеза MAX II

Предупреждения связаны с отсутствием физических назначений выводов в проекте для синтеза и с неиспользуемым входом `dir` в режиме `MODE = 0`; на корректность логики счётчика они не влияют.

ВСТАВИТЬ СЮДА РИСУНОК 4

Processing -> Compilation Report -> Flow Summary. Показать успешный статус, 0 ошибок и использование 4 из 240 логических элементов.

Рисунок 4 - Сводка компиляции проекта MAX II

ВСТАВИТЬ СЮДА РИСУНОК 5

Tools -> Netlist Viewers -> RTL Viewer. Перед снимком выполнить Fit in Window.

Рисунок 5 - RTL-схема проекта MAX II

9.2 Cyclone IV EP4CE10E22C8

Проект `quartus/cycloneiv/lab5_cycloneiv.qpf` подготовлен для платы из примера. Верхний уровень `board_top` содержит делители частоты, реверсивный поведенческий счётчик Грея, выходы на светодиоды и драйвер четырёх семисегментных индикаторов. Назначения выводов перенесены в QSF; перед прошивкой их необходимо сверить со схемой конкретной лабораторной платы. Полная компиляция в Quartus Prime 15.1 завершена успешно.

9.2.1 Верхний уровень платы

Файл `board_support.v` содержит делители разрешающих импульсов и драйвер индикации. Файл `tops.v` содержит верхние уровни `synth_top` и `board_top`. Оба файла приведены полностью.

Листинг 9 - Полный текст файла `src/board_support.v`

```
`timescale 1ns/1ps

// Отчет: раздел 9.2.1, листинг 9.
// Вспомогательные модули платы: делители разрешающих импульсов и индикация.

module clock_enable_divider #(
    parameter integer DIVISOR = 25000000
) (
    input wire clock,
    output reg enable = 1'b0
);
    reg [31:0] count = 32'd0;

    always @(posedge clock) begin
        if (count == DIVISOR - 1) begin
            count <= 32'd0;
            enable <= 1'b1;
        end else begin
            count <= count + 1'b1;
            enable <= 1'b0;
        end
    end
end
endmodule
```

```

module bit_display_driver (
    input wire      clock,
    input wire      scan_enable,
    input wire [3:0] value,
    output reg [7:0] indicator_select,
    output reg [7:0] segments
);
    reg [1:0] scan = 2'b00;
    reg current_bit;

    always @(posedge clock) begin
        if (scan_enable)
            scan <= scan + 1'b1;
    end

    always @(*) begin
        indicator_select = 8'b1111_1111;
        current_bit = 1'b0;
        case (scan)
            2'd0: begin indicator_select = 8'b1111_1110; current_bit = value[3]; end
            2'd1: begin indicator_select = 8'b1111_1101; current_bit = value[2]; end
            2'd2: begin indicator_select = 8'b1111_1011; current_bit = value[1]; end
            default: begin indicator_select = 8'b1111_0111; current_bit = value[0]; end
        endcase

        case (current_bit)
            1'b0: segments = 8'b1100_0000;
            1'b1: segments = 8'b1111_1001;
            default: segments = 8'b1111_1111;
        endcase
    end
endmodule

```

Листинг 10 - Полный текст файла src/tops.v

```

`timescale 1ns/1ps

// Отчет: разделы 9.1 и 9.2.1, листинг 10.
// synth_top - верхний уровень проекта MAX II.
// board_top - верхний уровень проекта Cyclone IV и лабораторной платы.
// Верхний уровень для получения отчета синтеза MAX II.
module synth_top (
    input wire      clock,
    input wire      dir,
    output wire [3:0] q
);
    counter_structural #(.MODE(0)) dut (
        .clock (clock),
        .enable (1'b1),
        .dir (dir),
        .q (q)
    );
endmodule

// Верхний уровень для платы Cyclone IV из примера одноклассника.
module board_top (
    input wire      clock,
    input wire      dir_button,
    output wire [3:0] leds,
    output wire [7:0] indicator_select,
    output wire [7:0] segments
);
    wire count_enable;
    wire scan_enable;
    wire [3:0] counter_value;
    wire dir;

    // Кнопки платы активны низким уровнем.
    assign dir = ~dir_button;

    clock_enable_divider #(.DIVISOR(25000000)) count_divider (
        .clock (clock),
        .enable (count_enable)
    )

```

```

);

clock_enable_divider #(.DIVISOR(5000)) scan_divider (
    .clock    (clock),
    .enable   (scan_enable)
);

counter_beh_function #(.MODE(2)) counter (
    .clock    (clock),
    .enable   (count_enable),
    .dir      (dir),
    .q        (counter_value)
);

assign leds = counter_value;

bit_display_driver display (
    .clock            (clock),
    .scan_enable      (scan_enable),
    .value            (counter_value),
    .indicator_select (indicator_select),
    .segments         (segments)
);
endmodule

```

Параметр	Результат
Статус	Успешно, 0 ошибок, 19 предупреждений
Логические элементы	118 из 10 320 (1 %)
Регистры	70
Выводы	22 из 92 (24 %)
Минимальный setup slack	14,706 нс
Минимальный hold slack	0,186 нс
Файл прошивки	output_files/lab5_cycloneiv.sof

Таблица 15 - Результаты синтеза Cyclone IV

Предупреждения относятся к постоянным уровням на неиспользуемых разрядах индикаторов, электрическим параметрам выводов и отсутствию задания неопределённости тактового сигнала. Все временные запасы положительны. Перед прошивкой требуется сверить распиновку с платой.

ВСТАВИТЬ СЮДА РИСУНОК 6

Processing -> Compilation Report -> Flow Summary. Показать успешный статус, 0 ошибок, 118 логических элементов и 70 регистров.

Рисунок 6 - Сводка компиляции проекта Cyclone IV

ВСТАВИТЬ СЮДА РИСУНОК 7

Tools -> Netlist Viewers -> RTL Viewer. Перед снимком выполнить Fit in Window.

Рисунок 7 - RTL-схема проекта Cyclone IV

ВСТАВИТЬ СЮДА РИСУНОК 8

Assignments -> Pin Planner. Показать сигналы clock, dir_button, leds, indicator_select, segments и номера выводов.

Рисунок 8 - Назначения выводов проекта Cyclone IV

ВСТАВИТЬ СЮДА РИСУНОК 9

Tools -> Programmer. Добавить lab5_cycloneiv.sof; при наличии платы показать Progress 100% Successful.

Рисунок 9 - Окно Programmer с файлом прошивки Cyclone IV

9.3 Экспериментальные исследования

После прошивки необходимо проверить отображение последовательности кодовых слов на светодиодах и индикаторах. В режиме dir = 0 счётчик должен проходить прямую последовательность Грея, а при dir = 1 - обратную. Для отчёта следует сфотографировать несколько последовательных состояний каждого направления и момент изменения направления.

Проверка	Ожидаемый результат
Счёт вверх	0000, 0001, 0011, 0010, 0110, ...
Счёт вниз	0000, 1000, 1001, 1011, 1010, ...
Смена направления	После переключения меняется порядок переходов без нарушения кода

Таблица 16 - План экспериментальной проверки

ВСТАВИТЬ СЮДА РИСУНОК 10

Фотография платы при отпущенной dir_button; должны читаться светодиоды и индикаторы.

Рисунок 10 - Экспериментальный счёт вверх в коде Грея

ВСТАВИТЬ СЮДА РИСУНОК 11

Фотография платы при нажатой dir_button; должны читаться светодиоды и индикаторы.

Рисунок 11 - Экспериментальный счёт вниз в коде Грея

ВСТАВИТЬ СЮДА РИСУНОК 12

Фотография или серия кадров, подтверждающая смену порядка переходов после переключения dir_button.

Рисунок 12 - Изменение направления счёта

10 Вывод

В работе выполнен структурный синтез четырёхразрядного счётчика варианта 9 на Т-триггерах. Для режима суммирования построен десятисостоянийный цикл кода Айкена 2421, а для режима вычитания - шестнадцатисостоянийный цикл отражённого кода Грея. Получены функции возбуждения, карты Карно и представления функций в базисах Пирса, Шеффера и Жегалкина.

Разработаны параметризованное структурное описание и четыре поведенческих варианта на Verilog HDL. Самопроверяющийся испытательный стенд охватывает суммирование, вычитание и смену направления. Проекты для MAX II EPM240T100C5 и Cyclone IV EP4CE10E22C8 успешно скомпилированы в Quartus Prime 15.1 без ошибок; сформированы файлы POF и SOF. После функционального моделирования и проверки на лабораторной плате в отчёт необходимо добавить временную диаграмму и фотографии экспериментальных состояний.

Приложение А Состав комплекта исходников

Файл	Размер байт
README.md	4033
FILE MAP.md	3408
QUARTUS_MODELSIM_GUIDE.md	11615
CHECKLIST AFTER QUARTUS.md	2228
src/t_ff.v	384
src/counter_structural.v	2311
src/counter_behavioral.v	11600
src/board_support.v	1638
src/tops.v	1802
tb/tb_all.v	6522
sim/run_sim.do	867
quartus/maxii/lab5_maxii.qpf	77
quartus/maxii/lab5_maxii.qsf	800
quartus/maxii/lab5_maxii.sdc	60
quartus/maxii/output_files/lab5_maxii.flow.rpt	6509
quartus/maxii/output_files/lab5_maxii.fit.summary	371
quartus/maxii/output_files/lab5_maxii.sta.summary	402
quartus/maxii/output_files/lab5_maxii.pin	15331
quartus/maxii/output_files/lab5_maxii.pof	7861
quartus/cycloneiv/lab5_cycloneiv.qpf	81
quartus/cycloneiv/lab5_cycloneiv.qsf	2546
quartus/cycloneiv/lab5_cycloneiv.sdc	60
quartus/cycloneiv/output_files/lab5_cycloneiv.flow.rpt	7179
quartus/cycloneiv/output_files/lab5_cycloneiv.fit.summary	617

Файл	Размер байт
quartus/cycloneiv/output_files/lab5_cycloneiv.sta.summary	952
quartus/cycloneiv/output_files/lab5_cycloneiv.pin	20832
quartus/cycloneiv/output_files/lab5_cycloneiv.sof	358727

Таблица А.1 - Файлы проекта

Приложение Б Соответствие листингов исходным файлам

Раздел	Листинг	Файл	Модуль	Назначение
8.1.1	1	src/t_ff.v	t_ff	Один Т-триггер
8.1.2-8.1.4	2	src/counter_structural.v	counter_structural	Три структурных режима
8.2.1	3	src/counter_behavioral.v	counter_beh_ternary	Тернарный оператор
8.2.2	4	src/counter_behavioral.v	counter_beh_case	Оператор case
8.2.3	5	src/counter_behavioral.v	counter_beh_if	Оператор if
8.2.4	6	src/counter_behavioral.v	counter_beh_function	Функция
8.3	7	tb/tb_all.v	tb_all	Самопроверяющийся стенд
8.3	8	sim/run_sim.do	сценарий	Запуск ModelSim
9.2.1	9	src/board_support.v	два модуля	Делители и индикация
9.1 и 9.2.1	10	src/tops.v	synth_top, board_top	Верхние уровни

Таблица Б.1 - Соответствие отчёта исходным файлам

Та же карта с дополнительными пояснениями находится в файле FILE_MAP.md внутри ZIP. В начале каждого Verilog-файла также указан номер раздела и листинга отчёта.

Приложение В Порядок запуска и оформления результатов

Подробная инструкция с названиями команд и разбором типовых ошибок находится в файле QUARTUS_MODELSIM_GUIDE.md. Ниже приведена последовательность обязательных действий.

Шаг	Действие	Результат
1	Распаковать ZIP в C:\lab5_smirnov	Путь без кириллицы
2	В ModelSim перейти в sim и выполнить do run_sim.do	Рисунки 2 и 3
3	Открыть lab5_maxii.qpf и выполнить Start Compilation	Рисунки 4 и 5
4	Открыть lab5_cycloneiv.qpf и выполнить Start Compilation	Рисунки 6-8
5	При наличии платы загрузить lab5_cycloneiv.sof	Рисунок 9
6	Проверить оба направления на плате	Рисунки 10-12
7	Заменить поля ВСТАВИТЬ СЮДА изображениями	Сохранить подписи
8	В Word нажать Ctrl+A и F9	Обновить содержание целиком

Таблица В.1 - Порядок завершения отчёта